# Performance Analysis of LLM Inference Engines on Deucalion CPU Nodes

Big Data Analysis Project Assignment #2

2025/2026

## 1 Introduction

This report follows Track A1, Single-Engine Deep Dive. The primary engine is `llama.cpp` served through the OpenAI-compatible `llama-server` HTTP interface. The experiments also include TurboQuant cache variants, the vanilla `llama.cpp` baseline, the same sweep on x86 nodes, and a vanilla BLAS comparison on ARM. These extra comparisons are used to explain the same A1 optimisation space against hardware differences rather than to claim a separate Track A2 study.

The parsed result set contains 1548 prompt-level summary rows and 1548 scaling rows. The raw sweep directories are `measurements/1253766-track-a1-server-sweep`, `measurements/1254633-track-a1-ser` and `measurements/1254637-track-a1-vanilla-blas-sweep`. The organized CSVs, plots, and this report are under `measurements/organized_a1_results`.

## 2 Methodology

The ARM runs used Deucalion `normal-arm` nodes with Fujitsu A64FX: 48 online CPUs, four NUMA nodes, one hardware thread per core, and SVE support. The x86 runs used `normal-x86` nodes with two AMD EPYC 7742 sockets: 128 online CPUs, eight NUMA nodes, one hardware thread per core, AVX2/FMA, and 256 GB class memory nodes as described in the assignment. The ARM assignment notes describe 32 GB class memory nodes. These capacity differences matter for headroom, but decode latency is primarily governed by model streaming and cache locality rather than capacity alone.

The benchmark client sends streaming HTTP requests and records time to first token (TTFT), time per output token (TPOT), output-token throughput, goodput, and sampled peak memory. Each configuration uses three measured trials after one warmup trial. Goodput uses the harness SLA of TTFT no larger than 2.0 s and TPOT no larger than 0.2 s. The prompt file contains at least ten prompts per category and marks the three mandatory assignment prompts. These sweeps used `LIMIT_PER_CATEGORY=3`, so the measured workload covers nine prompts per configuration: the three mandatory prompts plus six additional prompts. That is sufficient for the mandatory-prompt analysis below, but it is a partial run relative to the strict full-workload requirement of ten measured prompts per category. Generated text for the mandatory prompts is stored in `measurements/organized_a1_results/combined/mandatory_quality.csv`.

## 3 Metric Definitions

TTFT is reported in seconds and is the elapsed time from request submission to the first streamed output token. It mostly reflects prompt prefill, queueing, and startup overhead. TPOT is reported in milliseconds per generated token and is the steady-state decode latency. Throughput is reported as generated output tokens per second. Peak memory is reported as GiB from sampled VmHWM high-water resident memory. Goodput is a fraction from 0 to 1 using the benchmark SLA: TTFT no larger than 2.0 seconds and TPOT no larger than 200 milliseconds.

# 4 Results

| Sweep | Engine | Cache | Model | Threads | TTFT s | TPOT ms/token | Output t |
|---|---|---|---|---|---|---|---|
| arm_server | tq | turbo3 | model-1-mandatory | 24 | 0.525 | 145.3 | 5.461 |
| arm_server | tq | turbo4 | model-1-mandatory | 24 | 0.519 | 140.0 | 5.686 |
| arm_server | tq | turbo3 | model-3 | 24 | 11.304 | 164.4 | 0.254 |
| arm_server | tq | turbo4 | model-3 | 24 | 1.585 | 128.8 | 1.158 |
| arm_server | vanilla | f16 | model-1-mandatory | 24 | 0.555 | 130.4 | 6.130 |
| arm_server | vanilla | f16 | model-2 | 24 | 0.090 | 19.7 | 40.517 |
| arm_server | vanilla | f16 | model-3 | 24 | 1.772 | 127.7 | 3.545 |
| blas_server | vanilla-blis | f16 | model-1-mandatory | 24 | 0.543 | 130.4 | 6.142 |
| blas_server | vanilla-blis | f16 | model-2 | 24 | 0.089 | 21.1 | 38.120 |
| blas_server | vanilla-blis | f16 | model-3 | 24 | 1.777 | 127.8 | 3.540 |
| blas_server | vanilla-openblas-fujitsu | f16 | model-1-mandatory | 24 | 0.555 | 130.6 | 6.125 |
| blas_server | vanilla-openblas-fujitsu | f16 | model-2 | 24 | 0.088 | 19.6 | 40.679 |
| blas_server | vanilla-openblas-fujitsu | f16 | model-3 | 24 | 1.777 | 128.5 | 3.526 |
| x86_server | tq | turbo3 | model-1-mandatory | 32 | 0.191 | 158.9 | 5.924 |
| x86_server | tq | turbo4 | model-1-mandatory | 32 | 0.160 | 130.6 | 7.186 |
| x86_server | tq | turbo3 | model-3 | 32 | 0.293 | 93.9 | 2.940 |
| x86_server | tq | turbo4 | model-3 | 32 | 0.374 | 120.6 | 2.059 |
| x86_server | vanilla | f16 | model-1-mandatory | 32 | 0.165 | 122.7 | 7.520 |
| x86_server | vanilla | f16 | model-2 | 32 | 0.040 | 15.5 | 56.248 |
| x86_server | vanilla | f16 | model-3 | 32 | 0.468 | 129.0 | 4.690 |

Table 1: Baseline mandatory-prompt means with explicit units.

The main baseline pattern is that the x86 node delivers lower TPOT for the 8B mandatory model at the tested baseline, but it also uses a larger resident memory footprint. On ARM, vanilla `llama.cpp` and the BLAS-linked vanilla variants are competitive with or faster than the TurboQuant cache modes for the tested Q4 model. TurboQuant cache modes are still useful as an optimisation dimension because they change KV-cache representation, memory footprint, and decode behaviour, but in these measurements the dominant cost remains full model-weight streaming during autoregressive decode.

| Sweep | Engine | Cache | TTFT s | TPOT ms/token | Output tokens/s | Peak memory GiB |
|---|---|---|---|---|---|---|
| arm_server | tq | turbo3 | 0.525 | 145.3 | 5.461 | 4.83 |
| arm_server | tq | turbo4 | 0.519 | 140.0 | 5.686 | 4.87 |
| arm_server | vanilla | f16 | 0.555 | 130.4 | 6.130 | 5.25 |
| x86_server | tq | turbo3 | 0.191 | 158.9 | 5.924 | 7.99 |
| x86_server | tq | turbo4 | 0.160 | 130.6 | 7.186 | 8.03 |
| x86_server | vanilla | f16 | 0.165 | 122.7 | 7.520 | 8.42 |

Table 2: ARM versus x86 baseline comparison. The x86 wrapper used 32 threads; ARM used 24 threads.

Thread scaling shows diminishing returns. On ARM, performance improves when the thread count rises from one or two CMG-sized groups toward the 36–46 thread region, but 48 threads is not always best. On x86, 8–16 threads are often already strong; 128 threads can be worse because the request is spread across many NUMA domains and synchronization overhead dominates useful matrix-vector work. Concurrency behaves as expected for CPU serving: increasing simultaneous requests increases queueing and contention, so per-request TTFT and TPOT rise, and the goodput SLA degrades. Some high-concurrency BLAS runs returned no successful prompt rows, which should be treated as overload points rather than valid latency samples.

Context length mostly affects TTFT and memory because longer prompts increase prefill work and KV-cache size. Decode length has a smaller effect on TPOT; it mainly improves measurement stability because fixed startup overhead is spread over more generated tokens. The small SmolLM2 model is much faster and has a much smaller memory footprint. The Gemma model produced

several zero-token or weak-output rows, so those model-set results should be discussed as a quality or compatibility limitation rather than as a clean performance point.

| Engine | Threads | TTFT s | TPOT ms/token | Output tokens/s | Goodput fraction | Peak memory |
|---|---|---|---|---|---|---|
| vanilla-blis | 24 | 0.543 | 130.4 | 6.142 | 1.000 | 5.51 |
| vanilla-openblas-fujitsu | 24 | 0.555 | 130.6 | 6.125 | 1.000 | 5.47 |

Table 3: BLAS runtime comparison on ARM using vanilla llama.cpp.

The BLAS comparison shows OpenBLAS/Fujitsu and BLIS are very close for this server workload. That is consistent with decode being memory-traffic dominated: once the matrix-vector kernels are reasonably optimized, changing BLAS does not remove the need to stream the quantised weights for each output token.

## 5 Performance Model

For autoregressive decoding, a simple lower-bound model is

$$\mathrm{TPOT}_{pred} \geq \frac{\mathrm{model\_bytes}}{BW_{mem}}.$$

The intuition is that each output token requires reading a large fraction of the model weights. If the run is memory-bandwidth bound, TPOT should scale roughly with model size and inversely with attainable memory bandwidth. The repository does not currently contain a `memory_bandwidth.csv` STREAM measurement, so the validation table below uses the best observed effective model-streaming rate per sweep as an empirical bandwidth ceiling. This is weaker than an independent STREAM validation and should be replaced by measured TRIAD bandwidth before final submission.

| Sweep | Engine | Model | Model GiB | BW GB/s | Pred TPOT ms | Observed TPO |
|---|---|---|---|---|---|---|
| arm_server | tq | model-1-mandatory | 4.58 | 41.793 | 117.7 | 145.3 |
| arm_server | tq | model-1-mandatory | 4.58 | 41.793 | 117.7 | 140.0 |
| arm_server | tq | model-3 | 4.97 | 41.793 | 127.7 | 164.4 |
| arm_server | tq | model-3 | 4.97 | 41.793 | 127.7 | 128.8 |
| arm_server | vanilla | model-1-mandatory | 4.58 | 41.793 | 117.7 | 130.4 |
| arm_server | vanilla | model-2 | 0.25 | 41.793 | 6.5 | 19.7 |
| arm_server | vanilla | model-3 | 4.97 | 41.793 | 127.7 | 127.7 |
| blas_server | vanilla-blis | model-1-mandatory | 4.58 | 41.751 | 117.9 | 130.4 |
| blas_server | vanilla-blis | model-2 | 0.25 | 41.751 | 6.5 | 21.1 |
| blas_server | vanilla-blis | model-3 | 4.97 | 41.751 | 127.8 | 127.8 |
| blas_server | vanilla-openblas-fujitsu | model-1-mandatory | 4.58 | 41.751 | 117.9 | 130.6 |
| blas_server | vanilla-openblas-fujitsu | model-2 | 0.25 | 41.751 | 6.5 | 19.6 |

Table 4: Simple TPOT model validation. BW is calibrated from the best observed model-streaming rate in each sweep.

The model captures the correct first-order trend: the 8B Q4 model is much slower and much larger than SmolLM2, and the x86 node can sustain lower TPOT at the tested baseline. It breaks down where engine overhead, NUMA placement, thread scheduling, prompt processing, KV-cache layout, and failed/zero-token generations dominate. Ratios above one indicate overhead beyond a pure memory streaming lower bound. Ratios below or near one identify the rows used to calibrate the empirical bandwidth ceiling.

## 6 Bottleneck Analysis

The dominant bottleneck is memory traffic during decode, with secondary bottlenecks from thread coordination and NUMA locality. Evidence:

- TPOT tracks model size strongly: SmolLM2 has far lower memory use and much higher throughput than the 8B baseline.
- Decode length changes throughput less than model size or thread placement, meaning steady-state token generation is the limiting phase.
- High thread counts eventually hurt, especially on x86 at 128 threads, which is typical when memory bandwidth and cross-NUMA traffic saturate.
- BLAS library changes have small impact compared with architecture, model size, and threading.

The highest-impact optimisation would be to reduce bytes moved per generated token: smaller weight quantisation, better cache locality, NUMA-aware pinning, and speculative decoding or prompt batching where quality and latency budgets allow it. KV-cache quantisation helps memory footprint and long-context pressure, but it cannot fully remove the repeated model-weight traffic in decode.

# 7 Requirement Mapping

| Requirement | Evidence | Status |
|---|---|---|
| Track stated | Track A1 stated in scripts, README, and this report | Met |
| At least three models | Llama 3.1 8B Q4, SmolLM2 360M Q4, Gemma Q4 | Met |
| Mandatory baseline model | Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf | Met |
| Metrics | TTFT, TPOT, throughput, goodput, sampled VmHWM memory | Met |
| Three trials plus warmup | `trials=3`, `warmup_trials=1` in system logs | Met |
| Prompt dataset | JSON contains 10+ prompts per category and mandatory prompts | Met in |
| Measured prompt count | Runs used `LIMIT_PER_CATEGORY=3` | Partia |
| Optimisation dimensions | cache type, threads, concurrency, context, decode length, node architecture, BLAS | Met |
| Performance model | TPOT lower bound from model bytes and bandwidth | Partia |

Table 5: Assignment requirement mapping.

# 8 Conclusion

The measurements support a hardware-aware explanation of CPU-only LLM serving: decode is dominated by repeated movement of model weights, while prefill and long-context behaviour are reflected more strongly in TTFT and memory. The x86 EPYC node is faster for the tested baseline, but raw thread count is not enough: oversubscribing across NUMA domains can reduce throughput. On ARM A64FX, using most but not necessarily all cores is better than assuming maximum thread count is optimal. BLAS choice matters less than model size, memory traffic, and thread placement for the measured server workload.